

10.2 Reduction Trees

Average vs. Peak Parallelism

Because the resource requirement drops drastically at every step, there is a significant disparity between the maximum hardware needed and the average hardware used.

- **Peak Parallelism:** The highest number of simultaneous operations, which occurs at the first step ($N/2$). For $N = 1024$, the peak is 512.
- **Average Parallelism:** Calculated by dividing the total operations by the total time steps: $\frac{N-1}{\log_2(N)}$. For $N = 1024$, this is $\frac{1023}{10} = 102.3$.

This variation means that if hardware is built to handle the peak of 512 operations, most of those execution units will sit idle during the later steps of the reduction process.

10.5 Minimizing memory divergence

Assuming 4-byte elements and 128-byte hardware memory segments (which hold 32 elements each):

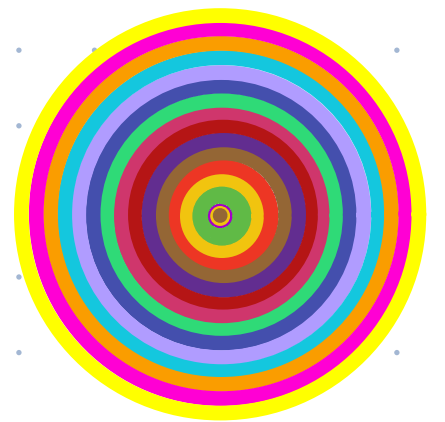
Warp 0 is responsible for below segments.

- **Segment 1** contains indices 0–31.
- **Segment 2** contains indices 32–63.

The access pattern for a single warp is as follows:

- **Iteration 1 (Stride 2):** 32 active threads access indices 0, 2, 4... up to 62. This spans both Segments 1 and 2 (2 requests).
- **Iteration 2 (Stride 4):** 16 active threads access indices 0, 4, 8... up to 60. This spans Segments 1 and 2 (2 requests).
- **Iteration 3 (Stride 8):** 8 active threads access indices 0, 8, 16... up to 56. This spans Segments 1 and 2 (2 requests).
- **Iteration 4 (Stride 16):** 4 active threads access indices 0, 16, 32, and 48. This spans Segments 1 and 2 (2 requests).
- **Iteration 5 (Stride 32):** 2 active threads access indices 0 and 32. This spans Segments 1 and 2 (2 requests).
- **Iteration 6:** 1 active thread accesses index 0. This falls entirely within Segment 1 (1 request).

The highest index accessed by the warp steadily decreases but remains in Segment 2 for exactly five iterations. Therefore, the hardware fetches exactly 2 memory segments until only a single thread remains active in the sixth iteration.



GPUs execute using the SIMT (Single Instruction, Multiple Threads) architecture. Threads in a warp do not fetch multiple elements independently; rather, all 32 threads in a warp execute the exact same instruction in lockstep before moving to the next instruction.

Therefore, the memory controller does not process "Thread 0 requesting elements 0 and 1" as a single unit. It processes the memory requests for the entire warp per instruction.

Instruction 1: Read `in[idx]`

- Thread 0 requests element 0

Thread 0 requests element 0.

- Thread 1 requests element 2.
- Thread 2 requests element 4.
- ...
- Thread 31 requests element 62.

For this single instruction, the warp requests a strided sequence of addresses (0, 2, 4... 62). These 32 requested elements span an address range of 64 contiguous integers (256 bytes). To serve this instruction, the hardware must fetch that entire 256-byte block from global memory (typically via two 128-byte transactions). Half of the data fetched (the odd indices) is entirely unused during the execution of this specific instruction.

Instruction 2: Read `in[idx + stride]` (where stride = 1)

- Thread 0 requests element 1.
- Thread 1 requests element 3.
- Thread 2 requests element 5.
- ...
- Thread 31 requests element 63.

For the second instruction, the warp requests the odd indices. The hardware accesses the same 256-byte span.

Perfect memory coalescing occurs only when contiguous threads access contiguous memory locations during a *single* instruction (e.g., Thread 0 reads 0, Thread 1 reads 1, Thread 2 reads 2). Because adjacent threads in this kernel request elements spaced two positions apart (`idx = threadIdx.x * 2`), the accesses are not perfectly coalesced, resulting in the wasted bandwidth described in the text.

Exercises

Q1.

$$\text{no. of warps} = \frac{1024}{64}$$

$$= 16$$

All of them will have divergence after first iteration.

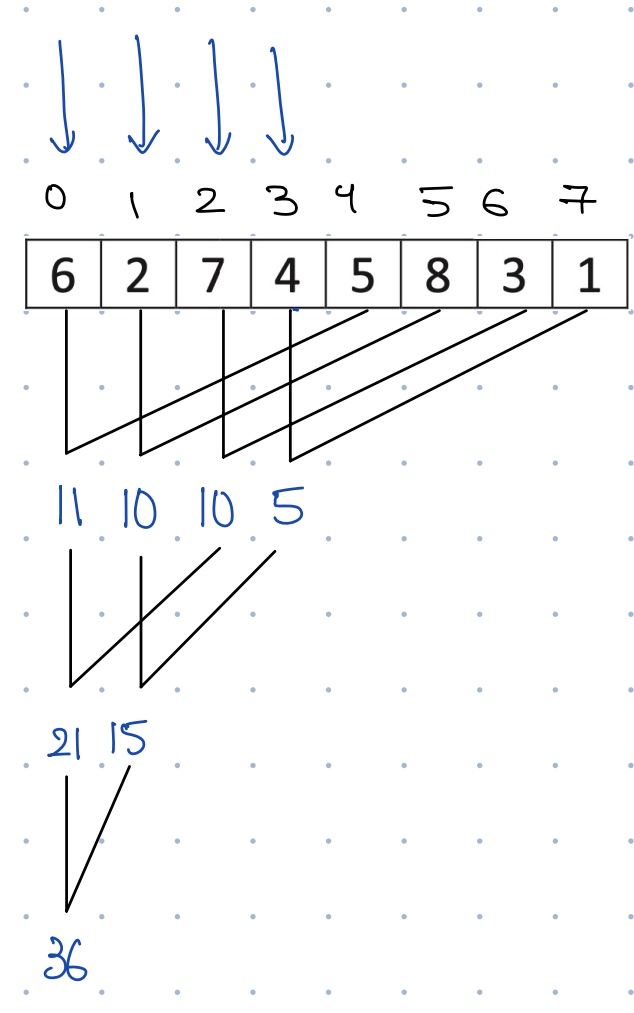
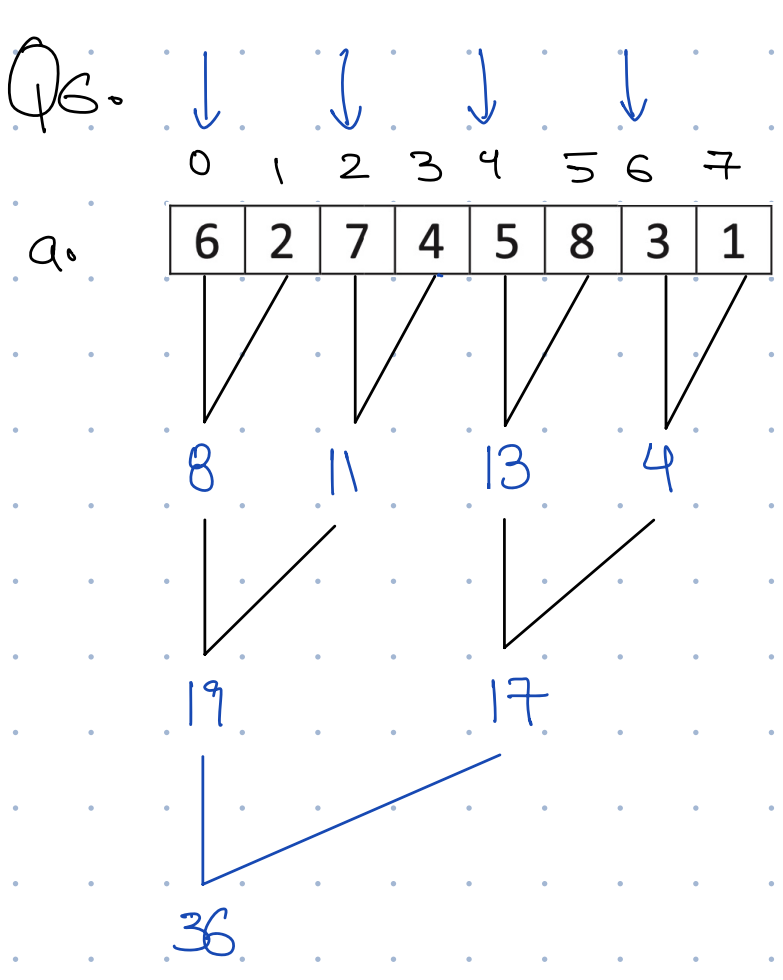
Q2.

$$\text{no. of warps} = \frac{1024}{64}$$

$$= 16$$

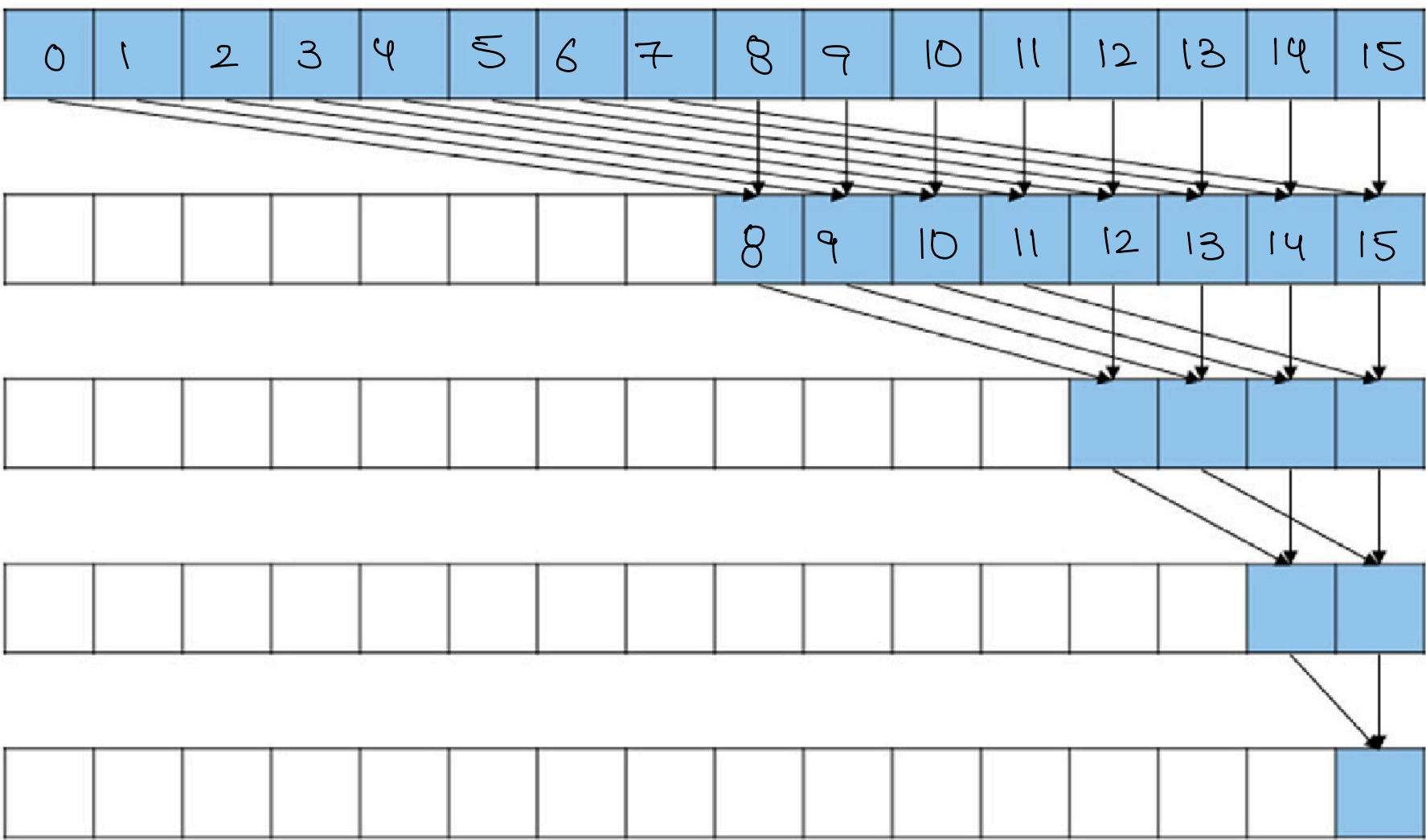
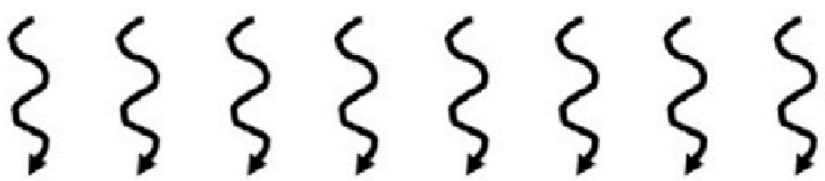
$$16 + 8 + 4 + 2 + 1$$

No warp will have divergence during fifth iteration.
Only after 5th iteration, only the first warp will have divergence.



Q3.

blockDim = 8



$$\text{blockDim} \cdot x - tx \geq \overset{4}{\text{stride}}$$
$$8 - \leq 4$$